Comprehensive Technical and Operations Guide

WhatsApp Hospital Appointment Chatbot Project

An explanation of the project architecture, the purpose of every file, the database, message flow, booking, rescheduling and cancellation, reminders, the admin dashboard, Meta integration, and operating procedures.

Project: `hospital_chatbot`
Path: `e:\4S\hospital_chatbot`
Guide prepared: July 16, 2026

Table of Contents

1. Overview and project status
2. Technology stack
3. Overall architecture
4. Directory structure
5. Startup and entry points
6. WhatsApp message journey
7. Complete booking flow
8. Rescheduling and cancellation
9. Working hours, slots, and holidays
10. Reminders
11. Voice messages
12. Safety and human handover
13. Database and relationships
14. Admin dashboard
15. Per-file guide
16. Environment variables
17. Operating commands
18. Current notes and limitations

1. Overview and Project Status

This project is an integrated backend for a hospital chatbot operating through the WhatsApp Cloud API. Patients can select a language, specialty or physician, view physician cards, choose a date and time, confirm a booking, and reschedule or cancel an appointment. All core operations are stored in PostgreSQL and displayed in a Streamlit administration dashboard.

> **Current supported execution path:** run `uvicorn app.main:app`. Root-level files such as `main.py` and `main_ai.py` are older experiments and are not the current production entry point.

Patient Interface

- WhatsApp text, buttons, lists, image cards, and an optional WhatsApp Flow.
- Arabic and English support.
- Booking, rescheduling, cancellation, inquiries, and receptionist handover.
- Voice messages with speech-to-text conversion.

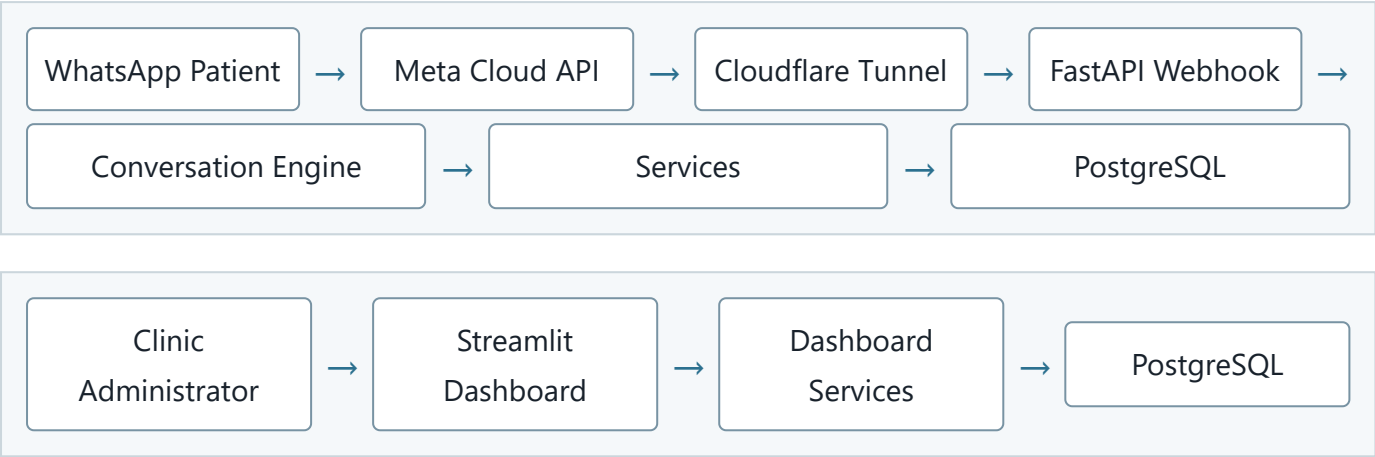
Administration Interface

- Streamlit dashboard.
- Clinic, department, physician, working-hours, and holiday management.
- Appointments, change history, conversations, and reminders.
- KPIs, usage, and estimated cost.

2. Technology Stack

Layer	Technology	Role
API	`FastAPI + Uvicorn`	WhatsApp webhook and experimental REST API.
Database	`PostgreSQL + SQLAlchemy 2`	Stores patients, physicians, appointments, conversations, and reminders.
Conversation management	`LangGraph`	Runs the message-persistence node followed by the state-processing node.
WhatsApp	`Meta WhatsApp Cloud API`	Sends and receives text, buttons, lists, cards, and Flow interactions.
Artificial intelligence	`OpenAI / Groq`	Resolves ambiguous choices and provides voice STT; the main flow uses explicit rules.
Voice	`Groq Whisper + edge-tts / gTTS`	Speech-to-text, with TTS utilities in the root-level files.
Admin dashboard	`Streamlit + Pandas`	Data administration, tables, and reports.
Tunnel	`Cloudflared`	Exposes localhost through an HTTPS address reachable by the Meta webhook.

3. Overall Architecture



Separation of Responsibilities

Directory	Responsibility
`app/api/routes`	Receives HTTP, extracts Meta data, and delegates work to services.
`app/graph`	Manages conversation state and the step-by-step interactive flow.
`app/services`	Business logic: bookings, slots, physicians, patients, reminders, WhatsApp, and handover.
`app/models`	SQLAlchemy table definitions and database relationships.
`app/core`	Configuration, localization, messages, time, safety, and prompts.
`dashboard`	Administration interface and presentation services.
`app/scripts`	Seeds demo data and runs reminders manually or on a schedule.

4. Directory Structure

```
hospital_chatbot/
├── app/
│   ├── main.py                Actual FastAPI entry point
│   ├── api/routes/            Webhook + Booking REST API
│   ├── graph/                 LangGraph conversation engine
│   ├── core/                  Config, i18n, safety, time
│   ├── services/              System logic and services
│   ├── models/                SQLAlchemy tables
│   ├── db/                    Connection and enums
│   ├── scripts/               Seed + reminders
│   ├── flows/                 WhatsApp Flow JSON
│   └── schemas/               Package reserved for expansion
├── dashboard/
│   ├── streamlit_app.py       Administration dashboard
│   └── ui_i18n.py             Administration UI localization
├── docs/                      Operations documentation and this guide
├── requirements.txt           Python dependencies
├── .env / .env.example        Secrets and environment configuration
├── README.md                  Brief introduction
├── voice_processor.py         Active STT implementation called by the service
├── main.py                    Unsupported legacy prototype
├── main_ai.py                 Standalone legacy Groq prototype
└── test.py                    Legacy local voice prototype
```

5. Startup and Entry Points

Supported API

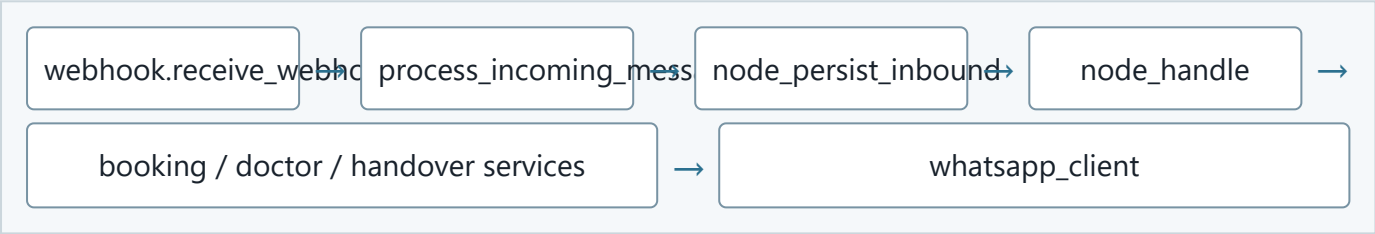
`app/main.py` creates the FastAPI application, runs `create_tables()` on startup, and includes `webhook_router` and `booking_router`.

- `GET /` : confirms that the server is running.
- `GET /webhook` : Meta verification handshake.
- `POST /webhook` : receives messages and delivery statuses.
- `GET /booking/departments` : specialty list.
- `GET /booking/doctors` : physician list.
- `GET /booking/doctors/{id}/slots` : available times.
- `POST /booking/confirm` : direct confirmation for tests without WhatsApp.
- `GET /docs` : FastAPI Swagger UI.

> **Database:** `app/db/database.py` builds an engine with connection pooling and provides `SessionLocal` and the `get_db` dependency. Creating tables with `Base.metadata.create_all` is suitable for development; Alembic migrations are preferable in production.

6. WhatsApp Message Journey Across Files

1. The patient sends a message to the WhatsApp Business number.
2. Meta sends JSON to `POST /webhook` in `app/api/routes/webhook.py`.
3. `receive_webhook` identifies text, button, list, Flow, audio, or a status update.
4. Text/button input is sent as a background task to `_handle_message`.
5. `_handle_message` opens a database session and calls `process_incoming_message`.
6. `conversation_engine.get_graph()` runs LangGraph.
7. `node_persist_inbound` stores the Conversation and Message.
8. `node_handle` reads conversation state and performs the appropriate step.
9. Services read/write PostgreSQL, then `whatsapp_client.py` sends the Meta response.



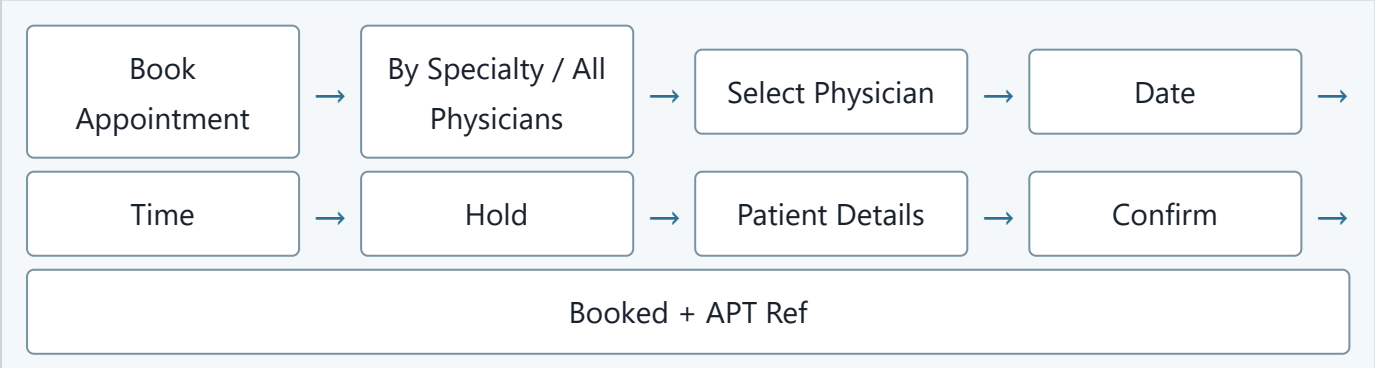
Conversation States

Stored in `Conversation.state` and defined in `ConversationState`:

`new` `language_select` `main_menu` `booking_doctor` `booking_date` `booking_slot` `patient_details` `confirming` `reschedule` `cancel` `handover` `closed`

The primary state is stored in the database, but in-progress booking draft details are held in the in-memory `_SESSION` dictionary inside the conversation engine, with recovery/restart behavior if they are lost after a reload.

7. Complete Booking Flow



7.1 Selecting a Specialty and Physician

- `intent_service.detect_intent` recognizes booking intent using rules first.
- `doctor_service.list_departments` retrieves departments; the department is the specialty shown to the patient.
- `doctor_service.list_doctors_by_department` retrieves physicians in the specialty.
- `doctor_card_payload` prepares the name, image, specialty, and biography.
- `whatsapp_client.send_doctor_cards` sends cards sequentially (up to five).
- If WhatsApp Flow is enabled, `flow_service` builds a carousel and radio list; it falls back to cards on failure.

7.2 Generating and Displaying Times

- `booking_service.get_available_dates_and_slots` validates the physician and releases expired holds.
- `slot_service.ensure_slots_for_doctor` generates slots from the physician's weekly hours.
- It excludes past times, minimum booking lead time, physician leave, and clinic holidays.
- `list_available_slots` returns only `available` slots for the same physician and clinic.
- `group_slots_by_date` groups them by date for WhatsApp lists.

7.3 Temporary Hold and Confirmation

1. `hold_slot` locks the slot row with `SELECT FOR UPDATE`.
2. It changes status from `available` to `held`, for three minutes by default.
3. It associates the hold with the WhatsApp number using a marker in `updated_by`.
4. The patient enters a name, and MRN/DOB if required by configuration.
5. The bot presents a summary and waits for “Confirm” or “Cancel.”
6. `confirm_booking` locks the row again and verifies that the time is available to that number.
7. It creates or updates Patient and creates an Appointment with `confirmed` status.
8. It changes Slot to `booked` and clears hold expiry.
9. It creates AppointmentHistory and Reminder records and closes the Conversation.
10. It commits and returns a reference such as `APT-...`.

Preventing Double Booking

> Protection is layered: the slot row is locked transactionally; `booked` slots are excluded from available times; and a partial unique index named `uq_appointment_active_slot` prevents more than one active Appointment for the same slot. With two simultaneous attempts, the second sees the booked status and fails.

8. Rescheduling and Cancellation

Rescheduling

1. `list_upcoming_appointments_for_phone` lists only confirmed appointments belonging to the patient's number.
2. The patient selects an appointment, then a new physician/date/time.
3. The new time is held for the same WhatsApp number.
4. `reschedule_appointment` verifies ownership and the cutoff.
5. The old slot becomes `available` and the new slot becomes `booked`.
6. The Appointment is updated and an AppointmentHistory record is added.
7. Old reminders are cancelled and reminders are created for the new time.

Cancellation

1. The patient selects an appointment and confirms cancellation.
2. `cancel_appointment` verifies the phone number, appointment, and confirmed status.
3. The Appointment becomes `cancelled`.
4. The Slot becomes `available`, making it visible to another patient.
5. AppointmentHistory is written and pending reminders are cancelled.

> Cancellation and rescheduling are subject to `CANCEL_CUTOFF_HOURS` and `RESCHEDULE_CUTOFF_HOURS`. If the appointment is closer than permitted, the request is transferred to reception.

9. Working Hours, Slots, and Holidays

Weekly Working Hours

`DoctorWorkingHours` defines weekday (0=Monday…6=Sunday), start and end time, appointment length, and maximum capacity. The dashboard saves hours through `admin_dashboard_service.upsert_working_hours`, then automatically generates slots.

Physician Leave

`DoctorUnavailability` closes a specific physician between two dates/times. When added through the dashboard, overlapping available times become `blocked`.

Clinic Holiday

`ClinicHoliday` closes the entire clinic. `block_available_slots_for_range` blocks available slots for every physician in the range.

Slot Statuses

Status	Meaning	Shown for booking?
`available`	Available.	Yes.
`held`	Temporarily reserved while details are entered.	No.
`booked`	Confirmed appointment.	No, until cancellation/rescheduling.
`blocked`	Holiday or closure.	No.
`expired`	Defined for future expansion; not currently used as a primary state.	No.

10. Reminders

- `create_booking_reminders` creates pending records before an appointment (24 and 2 hours by default).
- `process_due_reminders` retrieves due records and sends a localized WhatsApp message.
- Dry-run shows what would be sent without actually sending it.
- `cancel_pending_reminders` cancels pending records when an appointment is cancelled or rescheduled.
- Webhook statuses update a reminder to delivered/read/failed according to Meta receipts.
- The dashboard displays records and supports dry-run processing and retrying failures.

Manual execution uses `app/scripts/run_reminders.py`; the command can be scheduled through Task Scheduler or cron.

11. Voice Messages

1. The webhook detects `message.type = audio` and extracts `media_id`.
2. `voice_service.transcribe_whatsapp_audio` calls `voice_processor.process_whatsapp_audio`.
3. It downloads audio from Meta, sends it to Groq Whisper, and deletes the temporary file.
4. A voice-type Message and VoiceMessage metadata are stored in the database.
5. If confidence is low, the bot asks the user to record again or type.
6. If confidence is adequate, it displays the transcript and asks “Is this correct?” before continuing the flow.

> `test.py` contains an older EgypTalk local ASR prototype, while the active path in `app/services/voice_service.py` imports `voice_processor.py`, which uses Groq Whisper.

12. Safety and Reception Handover

- `core/safety.py` recognizes high-risk medical phrases and does not provide a diagnosis.
- It sends a language-appropriate emergency message and creates an urgent HandoverCase.
- `handover_service.create_handover` links the case to the conversation and patient, if available.
- It stores the handover reason, priority, open status, and a summary of recent messages.
- If a reception number is configured, reception can be notified through WhatsApp.
- Unexpected webhook errors also become high-priority handovers.

13. Database and Relationships

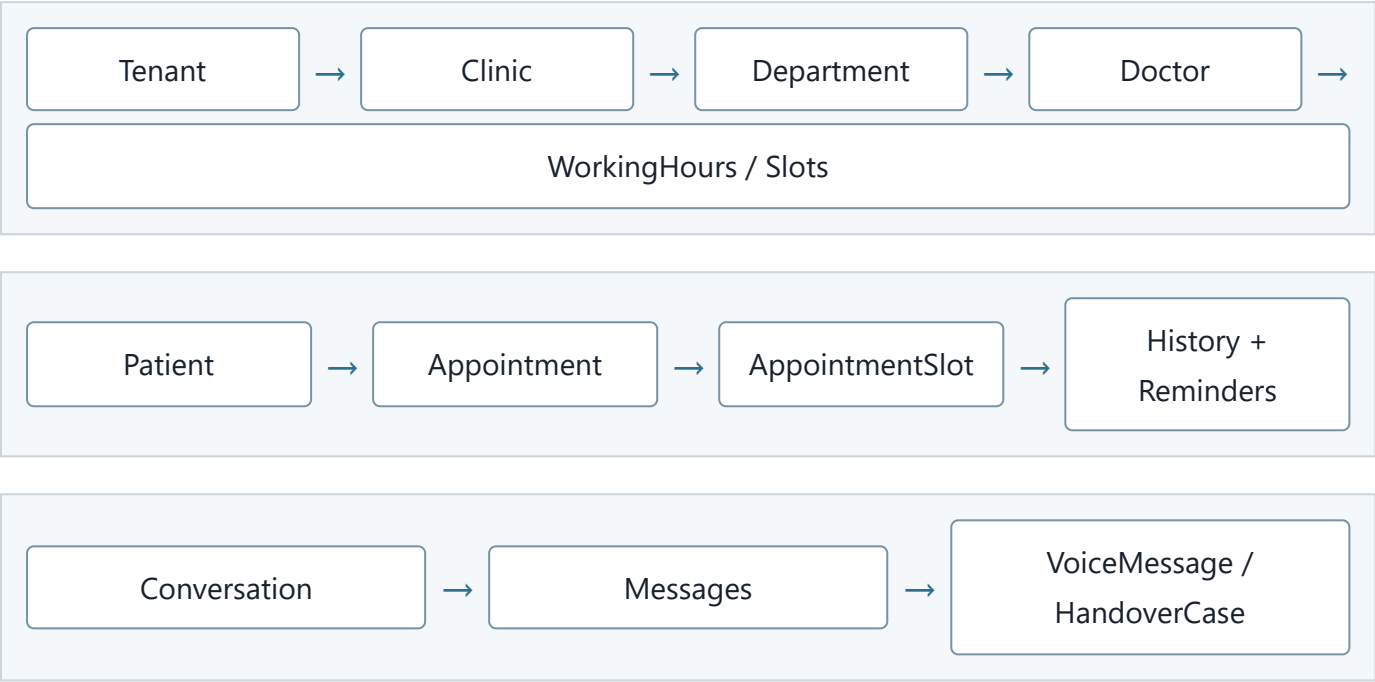


Table / Model	Purpose and Important Relationships
`Tenant`	Owning organization; has Clinics, Admins, and Doctors.
`Clinic`	Clinic linking departments, physicians, patients, appointments, conversations, and settings.
`Department`	Medical department/specialty within a clinic; contains physicians.
`ClinicHoliday`	Clinic-wide closure over a time range.
`Doctor`	Physician profile, image, specialty, and relationships to hours, slots, and appointments.
`DoctorLocation`	Physician locations/rooms and location hours.
`DoctorWorkingHours`	Recurring weekly schedule used to generate appointments.
`DoctorUnavailability`	Physician-specific leave/block.
`Patient`	WhatsApp number, name, MRN, DOB, appointments, and conversations.
`AppointmentSlot`	Specific physician time with status and hold expiry.
`Appointment`	Booking linking patient, physician, and slot, with reference and status.
`AppointmentHistory`	Audit trail for confirmation, rescheduling, cancellation, and slot changes.
`Conversation`	Conversation state, WhatsApp number, language, and current intent.
`Message`	Inbound/outbound log, delivery status, and Meta ID.
`VoiceMessage`	Audio details, transcript, provider, and confidence.

`Reminder`	Appointment-linked reminder, send time, status, and attempts.
`HandoverCase`	Human handover request with reason, priority, and summary.
`AdminUser`	Administration user associated with a Tenant.
`ConfigSetting`	Per-clinic key/value configuration.
`UsageLog`	Provider/operation/tokens/cost reporting.

Appointment Statuses

draft | held | confirmed | rescheduled | cancelled | completed | no_show | pending_handover

14. Admin Dashboard

`dashboard/streamlit_app.py` is a multi-screen Streamlit application. It opens a database session, calls `admin_dashboard_service` and `dashboard_service`, and displays results with Pandas.

Screen	Purpose
KPIs	Booking attempts, confirmations, cancellations, rescheduling, no-shows, handovers, voice, and cost.
Clinic setup	Clinic name, WhatsApp, timezone, language, address, general settings, and clinic holidays.
Departments	Add/edit specialties.
Doctors	Add/edit/disable/delete physicians, images, departments, and specialties.
Working hours	Weekly physician hours and individual leave; generate and block slots.
Doctor schedule	Free, Booked, Held, and Blocked times per physician, with booking details.
Appointments	List, status, cancellation, rescheduling, no-show, and history log.
Conversations	Conversations, complete messages, and handover cases.
Reminders	Records, send due, force demo, and retry failures.
Settings	Reminder hours, template name, escalation number, fallback, and consent.
Usage / costs	Usage and cost summary by provider/operation, plus recent logs.

`dashboard/ui_i18n.py` contains the Arabic/English UI text dictionary and screen ordering in `SCREENS`. Time is displayed in 12-hour format through `core/time_format.py`.

15. Guide to Every Project File

Active means part of the current execution path; **Legacy/supporting** means an earlier experiment or a file outside the primary runtime.

15.1 Root Files

File	Status	Purpose
`README.md`	Documentation	Very brief project and technology overview.
`requirements.txt`	**Active**	FastAPI, SQLAlchemy, LangGraph, Streamlit, OpenAI/Groq, and audio dependencies.
`.env`	**Active/secret**	Real keys and connection values; never publish or print them in documentation.
`.env.example`	Template	Required setting names without secrets.
`voice_processor.py`	**Active support**	WhatsApp audio download, Groq Whisper STT, plus TTS and audio upload/send utilities.
`main.py`	**Legacy**	Old prototype using in-memory sessions and hard-coded physician data; does not run the current system.
`main_ai.py`	**Legacy**	Standalone prototype using Groq function calling and mock physicians; not the real PostgreSQL system.
`test.py`	**Legacy**	EgypTalk local ASR + TTS experiment; not the current voice path.

15.2 Entry Point and API

File	Purpose
`app/main.py`	Creates FastAPI, creates tables at startup, and registers routers.
`app/api/routes/__init__.py`	Exports `booking_router` and `webhook_router`.
`app/api/routes/webhook.py`	Meta verification, message receipt, input-type extraction, background tasks, voice, Flow, receipts, and error handling.
`app/api/routes/booking.py`	REST endpoints for specialties, physicians, slots, and confirmation; useful for testing without WhatsApp.

15.3 Conversation Engine

File	Purpose
`app/graph/__init__.py`	Package interface exporting message processing and completion of physician selection from Flow.

app/graph/conversation_engine.py	System core: GraphState, session draft, language and menus, resolution, booking/rescheduling/cancellation states, LangGraph construction, and execution.

15.4 Core Files

File	Purpose
app/core/config.py	Reads .env and WhatsApp, AI, database, clinic, booking-policy, and reminder settings.
app/core/i18n_bot.py	Bilingual text layer, title and specialty localization, confirmations, cards, and menus.
app/core/messages.py	Core Arabic functions for welcome, menus, confirmation, rescheduling, cancellation, and reminders.
app/core/time_format.py	Standardizes Arabic/English 12-hour time, parsing, and dashboard time options.
app/core/safety.py	Detects high-risk medical phrases, provides emergency messages, and avoids medical advice.
app/core/prompts.py	**Legacy AI receptionist system prompt; the current guided flow does not directly depend on it.**
app/core/memory.py	**Legacy memory/deduplication dictionaries; the current engine has its own session and database logs.**
app/core/__init__.py	Package definition.

15.5 Business-Logic Services

File	Purpose and Key Functions
app/services/booking_service.py	Transactional booking: availability, hold, confirm, duplicate warning, upcoming appointments, reschedule, cancel, expired holds, and no-show.
app/services/slot_service.py	Generates slots from working hours, applies lead time and holidays, lists available slots, groups them, and locks with FOR UPDATE.
app/services/doctor_service.py	Department/physician search, soft matching, and physician card data preparation.
app/services/patient_service.py	Finds or creates a Patient by clinic and WhatsApp number, and updates details.
app/services/conversation_service.py	Creates/retrieves Conversation, updates language/state, and stores Message and Voice metadata.
app/services/whatsapp_client.py	Meta client: unified POST, text, buttons, lists, physician card(s), and WhatsApp Flow.

app/services/intent_service.py	Intent enum and rule-based detection, with LLM fallback when OpenAI is available.
app/services/choice_resolver.py	Resolves free text to option ID using normalization and rules, then LLM fallback.
app/services/flow_service.py	Prepares Doctor ImageCarousel data, downloads/compresses Base64 images, and parses Flow completion.
app/services/reminder_service.py	Creates, cancels, queries, and sends reminders; records wamid and status.
app/services/handover_service.py	Creates handovers, transcript snippets, priority and reception notifications, and lists cases.
app/services/voice_service.py	Thin adapter to voice_processor with a confidence heuristic.
app/services/dashboard_service.py	Queries for appointments, history, conversations, messages, reminders, physician schedules, and cancellation.
app/services/admin_dashboard_service.py	Clinic/department/physician/hours/leave CRUD, KPIs, usage, rescheduling, reminders, and no-show.
app/services/helpers.py	Generates appointment references.
app/services/__init__.py	Services package definition.

15.6 Database and Models

File	Purpose
app/db/database.py	Engine, pool, SessionLocal, Base, get_db, and create_tables.
app/db/enums.py	Booking, slot, reminder, conversation, message, priority, and language statuses.
app/db/__init__.py	Package definition.
app/models/__init__.py	Imports all models to register them in Base.metadata.
app/models/core_models.py	Tenant, Clinic, Department, and ClinicHoliday.
app/models/doctor_models.py	Doctor, DoctorLocation, DoctorWorkingHours, and DoctorUnavailability.
app/models/patient_models.py	Patient and its appointment/conversation relationships.
app/models/appointment_models.py	AppointmentSlot, Appointment, AppointmentHistory, constraints, and indexes.
app/models/conversation_models.py	Conversation, Message, and VoiceMessage.
app/models/reminder_models.py	Reminder schedule/status/Meta ID/retries.
app/models/handover_models.py	HandoverCase, reason, priority, assigned employee, and status.
app/models/admin_models.py	AdminUser and ConfigSetting.

app/models/usage_models.py	UsageLog for tokens, cost, provider, and operation.

15.7 Scripts, Flow, and Dashboard

File	Purpose
app/scripts/seed_demo.py	Creates Tenant/Clinic/Cardiology, four physicians with images and 30-minute hours, Admin, and Config. Current data may include dashboard additions.
app/scripts/run_reminders.py	Reminder CLI: limit, dry-run, and force.
app/scripts/__init__.py	Package definition.
app/flows/doctor_picker.json	Defines the WhatsApp Flow screen for images and selection.
app/flows/__init__.py	Package definition.
app/schemas/__init__.py	Empty/reserved package for future Pydantic schemas.
dashboard/streamlit_app.py	All dashboard pages, forms, display formatting, CSV export, login, and navigation.
dashboard/ui_i18n.py	Dashboard localization dictionary and screen list.
docs/WHATSAPP_FLOW_SETUP.md	Steps to create and publish a Flow in Meta and enable its variables.

16. Environment Variables

Variable	Purpose
`DATABASE_URL`	PostgreSQL URL.
`DEFAULT_TENANT_ID / DEFAULT_CLINIC_ID`	Single-clinic MVP context.
`VERIFY_TOKEN`	Meta webhook verification.
`WHATSAPP_TOKEN / PHONE_NUMBER_ID`	Sends messages through the Cloud API.
`USE_DOCTOR_FLOW / WHATSAPP_FLOW_ID / CTA / DRAFT`	Enables the optional WhatsApp Flow.
`OPENAI_API_KEY / GROQ_API_KEY / OPENAI_MODEL`	LLM and STT.
`SLOT_HOLD_MINUTES`	Temporary hold duration.
`BOOKING_WINDOW_DAYS / MIN_LEAD_HOURS`	Booking date window and minimum appointment lead time.
`CANCEL_CUTOFF_HOURS / RESCHEDULE_CUTOFF_HOURS`	Prevents late cancellation/rescheduling.
`DUPLICATE_APPOINTMENT_DAYS`	Similar-appointment warning period.
`ALLOW_OVERBOOKING`	Existing flag; in practice the system uses one slot per booking.
`REQUIRE_MRN / REQUIRE_DOB`	Makes patient details mandatory.
`REMINDER_HOURS_BEFORE`	For example, 24,2.
`CLINIC_DISPLAY_NAME / LOCATION / LOCATION_AR`	Clinic name and location in messages.
`RECEPTIONIST_WHATSAPP`	Human escalation number.
`ADMIN_DASHBOARD_PASSWORD`	Dashboard password; code default is admin123 for development.

> **Security:** never commit `.env` to Git or expose WhatsApp/OpenAI/Groq tokens in screenshots or logs.

17. Operating Commands

Install Dependencies and Seed Data

```
cd e:\4S\hospital_chatbot
pip install -r requirements.txt
python -m app.scripts.seed_demo
```

Run FastAPI

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Run the Admin Dashboard (Second Terminal)

```
streamlit run dashboard/streamlit_app.py
```

Run the Meta Tunnel (Third Terminal)

```
.\cloudflared tunnel --url http://127.0.0.1:8000
```

Reminders

```
python -m app.scripts.run_reminders --dry-run --force
python -m app.scripts.run_reminders --force --limit 2
```

It is normal for Uvicorn, Streamlit, or Cloudflared not to return to the command prompt; these are long-running server processes.

18. Current Technical Notes and Limitations

1. **Legacy path:** do not run `uvicorn main:app` for the current system; use `app.main:app`.
2. **Draft memory:** not all temporary booking details are database-backed; a restart during the flow may return the user to the menu.
3. **Hold ownership:** currently stored as a marker in `updated_by`, not a dedicated column; a `held_by_phone` column would be clearer in the future.
4. **Timezone:** many comparisons use naive datetimes and `utcnow()`, despite the Clinic timezone.
5. **30 minutes:** the demo assumes 30 minutes, but the dashboard permits changing slot length; keep it at 30 for the demo to work as intended.
6. **Reminder delivery:** current sending uses session messages/buttons; Meta templates are more appropriate outside the 24-hour service window.
7. **Alembic:** the library is installed but no migrations directory is visible; production needs migrations instead of relying only on create_all.
8. **Authentication:** the dashboard uses a simple environment-password comparison, not a complete hashed-password/session system.
9. **Tests:** there is no organized test suite for the current path; `test.py` is an audio prototype, not an automated test.
10. **Legacy cleanup:** moving old files to an archive directory would reduce the risk of launching the wrong application.

Summary

The current design cleanly separates HTTP, the conversation engine, business logic, and the database. The key path is `webhook.py → conversation_engine.py → services → models/PostgreSQL → whatsapp_client.py`. The Streamlit dashboard uses the same database, so bookings, rescheduling, and cancellations appear immediately in administration screens.